



**Politechnika
Śląska**

PROJEKT INŻYNIERSKI

System wspomagający inteligentne składanie zestawów komputerowych

Piotr KUPCZYK

Nr albumu: 305198

Kierunek: Teleinformatyka S1

Specjalność: Brak

PROWADZĄCY PRACĘ

dr hab. inż. Bożena Małysiak-Mrozek

KATEDRA Systemów Rozproszonych i Urządzeń Informatyki

Wydział Automatyki, Elektroniki i Informatyki

Gliwice 2025

Tytuł pracy

System wspomagający inteligentne składanie zestawów komputerowych

Streszczenie

(Streszczenie pracy – odpowiednie pole w systemie APD powinno zawierać kopię tego streszczenia.)

Słowa kluczowe

(2-5 słów (fraz) kluczowych, oddzielonych przecinkami)

Thesis title

A system supporting intelligent assembly of computer sets

Abstract

(Thesis abstract – to be copied into an appropriate field during an electronic submission – in English.)

Key words

(2-5 keywords, separated by commas)

Spis treści

1	Wstęp	1
1.1	Wprowadzenie w problem	1
1.2	Osadzenie problemu w dziedzinie	1
1.3	Cel pracy	1
1.4	Zakres pracy	2
1.5	Zwięzła charakterystyka rozdziałów	3
1.6	Wkład autora	4
2	Analiza problemu oraz przegląd dostępnych rozwiązań	5
2.1	Sformułowanie problemu	5
2.1.1	Istota kompatybilności	5
2.1.2	Niska dostępność narzędzi do weryfikacji kompatybilności	5
2.2	Analiza istniejących rozwiązań w najpopularniejszych sklepach na polskim rynku	6
2.2.1	Morele.net	6
2.2.2	X-kom.pl	6
2.3	Porównanie istniejących rozwiązań z rozwiązaniem inżynierkim autora . . .	6
3	Wymagania i narzędzia	9
3.1	Wymagania funkcjonalne	10
3.2	Wymagania нефункционалне	11
3.3	Przypadki użycia	12
3.4	Narzędzia i technologie	13
3.4.1	Backend	13
3.4.2	Frontend	13
3.4.3	Modelowanie i narzędzia wspomagające	13
3.5	Metodyka pracy	14
4	Specyfikacja wewnętrzna	15
4.1	Представление идеи системы	15
4.2	Architektura systemu	15

4.2.1	Warstwa frontendowa	15
4.2.2	Warstwa backendowa	15
4.2.3	Warstwa danych i infrastruktury	15
4.3	Opis struktur danych	15
4.3.1	Model danych - ERD	16
4.3.2	Opis najważniejszych encji	16
4.4	Komponenty i moduły systemu	16
4.4.1	Moduł kompatybilności podzespołów	16
4.4.2	Moduł logiki rozmytej	16
4.4.3	Moduł zarządzania koszykiem i zamówieniami	16
4.5	Najważniejsze algorytmy	16
4.6	Diagramy UML	16
4.7	Szczegóły implementacji wybranych elementów	16
5	Weryfikacja i walidacja	17
5.1	Cel weryfikacji systemu	18
5.2	Metody testowania	18
5.3	Organizacja eksperymentów	18
5.4	Przypadki testowe	18
5.5	Wyniki testów	18
5.6	Wykryte i usunięte błędy	19
5.7	Ocena poprawności działania systemu	19
6	Podsumowanie i wnioski	21
6.1	Realizacja celów pracy	21
6.2	Ocena otrzymanych rezultatów	21
6.3	Trudności napotkane podczas realizacji	21
6.4	Możliwe kierunki dalszego rozwoju	21
6.5	Wnioski końcowe	21
	Bibliografia	23
	Spis skrótów i symboli	27
	Źródła	29
	Lista dodatkowych plików, uzupełniających tekst pracy	31
	Spis rysunków	33
	Spis tabel	35

Rozdział 1

Wstęp

1.1 Wprowadzenie w problem

Sklepy komputerowe zazwyczaj posiadają swój własny sklep internetowy, oferujący możliwość zakupów części indywidualnie, jak i gotowych zestawów PC. Odpowiedni dobór komponentów jest najistotniejszą częścią procesu projektowania własnego stanowiska, a pomimo tego najwięksi sprzedawcy nie oferują możliwości sprawdzania kompatybilności wybranych produktów. Dlatego projekt ma na celu stworzenie prostej i intuicyjnej strony internetowej, która zaoferuje przystępną formę weryfikacji poprawności wyboru komponentów z katalogu oraz finalnie dodanie kompatybilnego zestawu do koszyka użytkownika.

1.2 Osadzenie problemu w dziedzinie

Projekt osadzony jest w dziedzinie inżynierii oprogramowania, projektowania baz danych oraz systemów e-commerce. Praca obejmuje zagadnienia relacyjności baz danych, budowy warstwy komunikacyjnej backendu, implementacji logiki aplikacji pod kątem biznesowym oraz wdrożenie interfejsu frontendowego.

1.3 Cel pracy

Celem pracy jest zaprojektowanie i zaimplementowanie aplikacji webowej, która będzie pełnić funkcję inteligentnego kreatora zestawów komputerowych. System będzie dawać użytkownikowi możliwość wybrania dowolnych produktów z katalogu, sprawdzenie czy aktualne zamówienie jest wzajemnie kompatybilne, a gdy weryfikacja zakończy się pomyślnie, umożliwi realizację zamówienia.

Idące za tym cele to

- stworzenie relacyjnej bazy danych, odpowiadającej potrzebom sklepu internetowego,
- zaprojektowanie i implementacja backendu, odpowiedzialnego za komunikację i logikę aplikacji,
- wdrożenie przejrzystego frontendu, będącego interfejsem klienta,
- stworzenie grupy zasobów w środowisku chmurowym Azure (baza danych jako storage, aplikacja jako kontener).

1.4 Zakres pracy

Zakres pracy nad projektem obejmuje:

- Projekt relacyjnej bazy danych w Oracle SQL Developer.
- Utworzenie testowego serwera PostgreSQL, wprowadzenie gotowych danych o produktach do bazy danych,
- Stworzenie użytkowników oraz nadanie odpowiednich uprawnień każdemu z nich,
- Implementację backendu w języku Python, implementacja API z wykorzystaniem frameworka FastAPI, umożliwiając integrację z bazą danych w celu zarządzania produktami i zamówieniami,
- Projekt interfejsu użytkownika, stworzenie frontendu używając HTML, CSS oraz JavaScript,
- Projekt kreatora zestawów komputerowych, zaimplementowanego z uwzględnieniem logik rozmytej, używając Reacta, Typescript, Node.js,
- Migrację gotowej aplikacji z lokalnego serwera testowego PostgreSQL do produkcyjnej chmury Azure, utworzenie grupy zasobów.

1.5 Zwięzła charakterystyka rozdziałów

- **Rozdział 1** - zawiera wprowadzenie do tematyki pracy, przedstawia problem nieintuicyjnego doboru kompatybilnych podzespołów komputerowych, definiuje cel i zakres pracy oraz opisuje autorski wkład w projekt.
- **Rozdział 2** - obejmuje analizę tematu, w tym istotę kompatybilności sprzętowej, przegląd istniejących rozwiązań wykorzystywanych przez popularne sklepy komputerowe oraz porównanie ich funkcjonalności z zaprojektowanym systemem.
- **Rozdział 3** - przedstawia wymagania funkcjonalne i нефunkcjonalne systemu, przypadki użycia, zastosowane narzędzia i technologie oraz metodykę pracy przyjętą podczas implementacji.
- **Rozdział 4** - stanowi część projektową, zawierającą architekturę systemu, model danych wraz z diagramem ERD, opis struktury bazy danych oraz omówienie kluczowych modułów, takich jak moduł weryfikacji kompatybilności i logiki rozmytej.
- **Rozdział 5** - prezentuje proces weryfikacji i walidacji systemu, zawierając opis sposobu testowania, organizacji eksperymentów, przypadków testowych oraz analizę wykrytych błędów i wyników testów.
- **Rozdział 6** - zawiera podsumowanie uzyskanych rezultatów, ocenę stopnia realizacji celów pracy oraz wskazuje możliwe kierunki dalszego rozwoju systemu.

1.6 Wkład autora

Autor samodzielnie opracował wszystkie elementy projektu, zaczynając od zaplanowania funkcjonalności oraz wyboru technologii. Następnym krokiem było zaprojektowanie relacyjnej bazy danych w Oracle SQL Developer, gdzie utworzył on testowy serwer PostgreSQL wraz z użytkownikami oraz nadał odpowiednie uprawnienia każdemu z nich. Przy użyciu frameworku FastAPI przygotowany został backend w języku Python, odpowiadający za komunikację z bazą danych oraz za logikę całej aplikacji, umożliwiając weryfikację kompatybilności zestawów komputerowych. Autor utworzył interaktywny frontend, który umożliwia graficzne korzystanie z aplikacji, zapewniając łatwość użytkowania zarówno dla doświadczonych, jak i początkujących inżynierów sprzętu, poprzez stosowanie logiki rozmytej. Korzystając z akademickiej subskrypcji Microsoft Azure autor używając otrzymanych kredytów samodzielnie realizuje hosting, administrację, testowanie CI/CD, konteneryzację oraz wirtualizację aplikacji webowej, czyniąc ją w pełni używalną w publicznej sieci internet.

Rozdział 2

Analiza problemu oraz przegląd dostępnych rozwiązań

2.1 Sformułowanie problemu

2.1.1 Istota kompatybilności

Na rynku znajduje się szeroka gama komponentów komputerowych, która nieustannie poszerza się o nowe produkty. Różnice pomiędzy poszczególnymi modelami dotyczą chociażby wymiarów fizycznych, dostępnych złączy, technologii socketów, minimalnej mocy zasilacza, aż do wymogów software’owych. Odpowiedni dobór części jest pierwszym krokiem do realizacji poprawnie funkcjonującego stanowiska komputerowego, gdyż dowolna pomyłka może prowadzić do braku możliwości fizycznego montażu lub niewłaściwej i nieciągłej pracy urządzenia. Nawet minimalne niezgodności mogą być przyczyną poważnej awarii, która może przyczynić się do strat finansowych, spowodowanych utratą danych, przerwaniem trwałości usług lub całkowitym uszkodzeniem sprzętu.

2.1.2 Niska dostępność narzędzi do weryfikacji kompatybilności

Aktualnie na polskim rynku sklepów komputerowych brakuje narzędzi umożliwiających użytkownikowi samodzielną i automatyczną weryfikację kompatybilności wybranych podzespołów. Większość sprzedawców oferuje wyłącznie gotowe zestawy komputerowe, które zapewniają pełną funkcjonalność, jednak często nie umożliwiają ich łatwej rozbudowy ani wymiany poszczególnych elementów na nowsze modele. W efekcie zaprojektowanie stanowiska w pełni odpowiadającego potrzebom użytkownika staje się utrudnione i może wymagać konsultacji ze specjalistą, co generuje dodatkowe koszty oraz wydłuża czas realizacji zamówienia.

2.2 Analiza istniejących rozwiązań w najpopularniejszych sklepach na polskim rynku

2.2.1 Morele.net

Spośród przeanalizowanych platform sprzedażowych polskich sprzedawców, pod względem technicznym najlepiej zaprojektowanym rozwiązaniem okazał się konfigurator zestawów PC dostępny w sklepie Morele[1]. Narzędzie oferuje intuicyjny interfejs oraz możliwość wyboru szerokiej gamy podzespołów. Kluczowym ograniczeniem jest jednak brak automatycznej weryfikacji kompatybilności, gdzie w przypadku błędnie skonfigurowanego zestawu jedynie pracownik sklepu może ewentualnie skontaktować się z użytkownikiem. W praktyce znacząco opóźnia to proces realizacji zamówienia i niesie ryzyko zakupu podzespołów, które nie będą ze sobą wzajemnie kompatybilne. Dodatkowo konfigurator nie wykorzystuje logiki rozmytej przy sugerowaniu komponentów.

2.2.2 X-kom.pl

Drugim pod względem funkcjonalności rozwiązaniem jest konfigurator oraz oferta gotowych zestawów sklepu X-kom[2]. Platforma udostępnia czytelny interfejs oraz rekomendowane zestawy komputerowe, które zazwyczaj umożliwiają późniejszą rozbudowę. W porównaniu do Morele[1], użytkownik ma tutaj jeszcze mniejszą możliwość samodzielnego projektowania konfiguracji, ponieważ wybór ogranicza się do kilku lub kilkunastu proponowanych zestawów. Brak szczegółowej analizy potrzeb użytkownika oraz ograniczona liczba wariantów utrudniają złożenie w pełni dostosowanego zamówienia. Podobnie jak w przypadku Morele, sklep nie wykorzystuje logiki rozmytej w mechanizmach komponowania zestawów.

2.3 Porównanie istniejących rozwiązań z rozwiązaniem inżynierkim autora

Pierwszą i najważniejszą funkcjonalnością wyróżniającą opracowane rozwiązanie jest zastosowanie rozszerzonych mechanizmów weryfikacji kompatybilności podzespołów, których brakuje w analizowanych sklepach internetowych. System wykorzystuje nie tylko klasyczne metody porównywania parametrów technicznych, lecz również elementy logiki rozmytej pozwalającej na bardziej elastyczną ocenę dopasowania podzespołów do oczekiwań użytkownika.

Aplikacja dokonuje szczegółowej analizy technicznej wszystkich wybranych komponentów, co przekłada się na sprawdzanie i weryfikowanie:

- zgodności procesora i płyty głównej pod względem zastosowanego gniazda socket'u,
- typu pamięci RAM oraz jej zgodności ze standardem płyty głównej,
- maksymalnej obsługiwanej częstotliwość i pojemność modułów pamięci,
- parametrów energetycznych, takich jak zapotrzebowanie na moc (TDP) oraz wydajności i maksymalnej dostarczanej mocy zasilacza,
- wymiarów i formatów kart graficznych oraz obudowy,
- zgodności chłodzenia procesora z wybranym gniazdem i limitem wysokości w obudowie.

W odróżnieniu od popularnych serwisów takich jak Morele.net[1] czy X-kom[2], które nie oferują pełnej automatycznej weryfikacji kompatybilności, system analizuje wszystkie istotne zależności pomiędzy podzespołami. Dzięki temu użytkownik nie jest narażony na ryzyko zakupu części wzajemnie niekompatybilnych, a proces konfiguracji zestawu przebiega szybciej, sprawniej i bez konieczności konsultacji ze specjalistą. Zastosowanie logiki rozmytej pozwala ponadto na ocenę zestawów zarazem pod kątem poprawności technicznej, lecz także ich dopasowania do ogólnych kategorii takich jak budżet, wydajność czy energooszczędność.

Rozdział 3

Wymagania i narzędzia

Celem niniejszego rozdziału jest przedstawienie zestawu wymagań funkcjonalnych i нефункциональных projektowanego systemu, omówienie narzędzi oraz technologii wykorzystanych podczas implementacji, a także zaprezentowanie metodyki pracy przyjętej podczas realizacji projektu inżynierskiego. Rozdział stanowi podstawę do dalszych etapów projektowania, implementacji oraz testowania aplikacji.

3.1 Wymagania funkcjonalne

Wymagania funkcjonalne określają zachowania oraz funkcje, jakie system powinien realizować. Na podstawie analizy problemu oraz specyficznych potrzeb użytkowników wydzielono następujący zbiór wymagań:

- Rejestracja oraz logowanie użytkowników.
- Przeglądanie katalogu produktów sklepu komputerowego.
- Dodawanie wybranych komponentów do kreatora zestawu PC.
- Automatyczna weryfikacja kompatybilności podzespołów, obejmująca między innymi:
 - zgodność gniazda procesora,
 - typ pamięci RAM,
 - kompatybilność częstotliwości i pojemności pamięci,
 - TDP procesora oraz wymagania energetyczne,
 - dopasowanie komponentów do obudowy,
 - kompatybilność układów chłodzenia.
- Wykorzystanie logiki rozmytej do klasyfikacji zestawów pod konkretne kategorie (np. budżetowy, wydajny, energooszczędny).
- Generowanie gotowych konfiguracji na podstawie wybranych komponentów.
- Dodawanie produktów oraz pełnych zestawów do koszyka.
- Modyfikacja zawartości koszyka oraz finalizacja zamówienia.

3.2 Wymagania нефункционалне

Wymagania нефункционалне описују влаџиwości jakościowe systemu, które wpływają na komfort użytkowania, bezpieczeństwo oraz niezawodność.

- **Wydaјność:** czas odpowiedzi API nie powinien przekraczać 1 sekundy przy standardowym obciążeniu.
- **Skalowalność:** system powinien umożliwiać uruchomienie w środowisku chmurowym Microsoft Azure.
- **Bezpieczeństwo:** hasła użytkowników muszą być przechowywane w formie zaszyfrowanej (bcrypt).
- **Niezawodność:** system powinien obsługiwać błędy kompatybilności w sposób kontrolowany i komunikatywny.
- **Użyteczność:** interfejs użytkownika powinien być przejrzysty, responsywny i intuicyjny.
- **Przenośność:** frontend uruchamiany w dowolnej nowoczesnej przeglądarce; backend kompatybilny z systemami Linux i Windows.

3.3 Przypadki użycia

Najważniejsze przypadki użycia systemu obejmują:

- „Użytkownik rejestruje nowe konto”
- „Użytkownik loguje się do systemu”
- „Użytkownik przegląda dostępne komponenty”
- „Użytkownik buduje własny zestaw PC”
- „System sprawdza kompatybilność elementów”
- „Użytkownik dodaje zestaw do koszyka”
- „Użytkownik realizuje zamówienie”

Diagramy przypadków użycia UML:

3.4 Narzędzia i technologie

Realizacja systemu wymagała zastosowania nowoczesnych technologii backendowych, frontendowych oraz narzędzi modelowania. Wszystkie przedstawiono poniżej.

3.4.1 Backend

- **Python 3** - język implementacji warstwy serwerowej.
- **FastAPI** - framework umożliwiający tworzenie wydajnego REST API.
- **SQLAlchemy** - ORM realizujący komunikację z bazą danych.
- **PostgreSQL** - relacyjna baza danych przechowująca informacje o produktach i zamówieniach.
- **psycopg2** - sterownik PostgreSQL wykorzystywany do komunikacji z bazą.
- **Pydantic** - walidacja modeli danych wykorzystywana przez FastAPI.
- **Uvicorn** - serwer ASGI obsługujący zapytania HTTP.

3.4.2 Frontend

- **React** - biblioteka wykorzystywana do tworzenia interaktywnych interfejsów użytkownika.
- **TypeScript** - rozszerzenie JavaScript umożliwiające statyczne typowanie.
- **Vite** - bundler odpowiedzialny za budowanie i szybkie uruchamianie aplikacji.
- **Tailwind CSS** - framework ułatwiający projektowanie responsywnego interfejsu.
- **Node.js** - środowisko wykonawcze dla narzędzi JavaScript.
- **npm** - menedżer pakietów odpowiedzialny za instalację zależności.
- **ESLint** - narzędzie do analizy i kontroli jakości kodu TypeScript i JavaScript.

3.4.3 Modelowanie i narzędzia wspomagające

- **Oracle SQL Developer Data Modeler** - narzędzie użyte do projektowania modelu ERD bazy danych.
- **Git** - system kontroli wersji.
- **GitHub** - repozytorium kodu oraz platforma do zarządzania wersjami projektu.
- **Microsoft Azure** - środowisko do wdrożenia i hostowania bazy danych.

3.5 Metodyka pracy

Prace nad projektem realizowano iteracyjnie, zgodnie z podejściem zbliżonym do metodyki Agile. Każda iteracja obejmowała:

- analizę oraz doprecyzowanie wymagań,
- projekt części funkcjonalnych systemu,
- implementację nowych modułów backendu i frontendu,
- testy poprawności działania,
- refaktoryzację oraz optymalizację kodu.

Podejście iteracyjne pozwoliło na równoległe rozwijanie bazy danych, backendu oraz interfejsu użytkownika, a także na bieżącą reakcję na problemy i dopasowywanie projektu do napotkanych wyzwań.

Rozdział 4

Specyfikacja wewnętrzna

4.1 Przedstawienie idei systemu

4.2 Architektura systemu

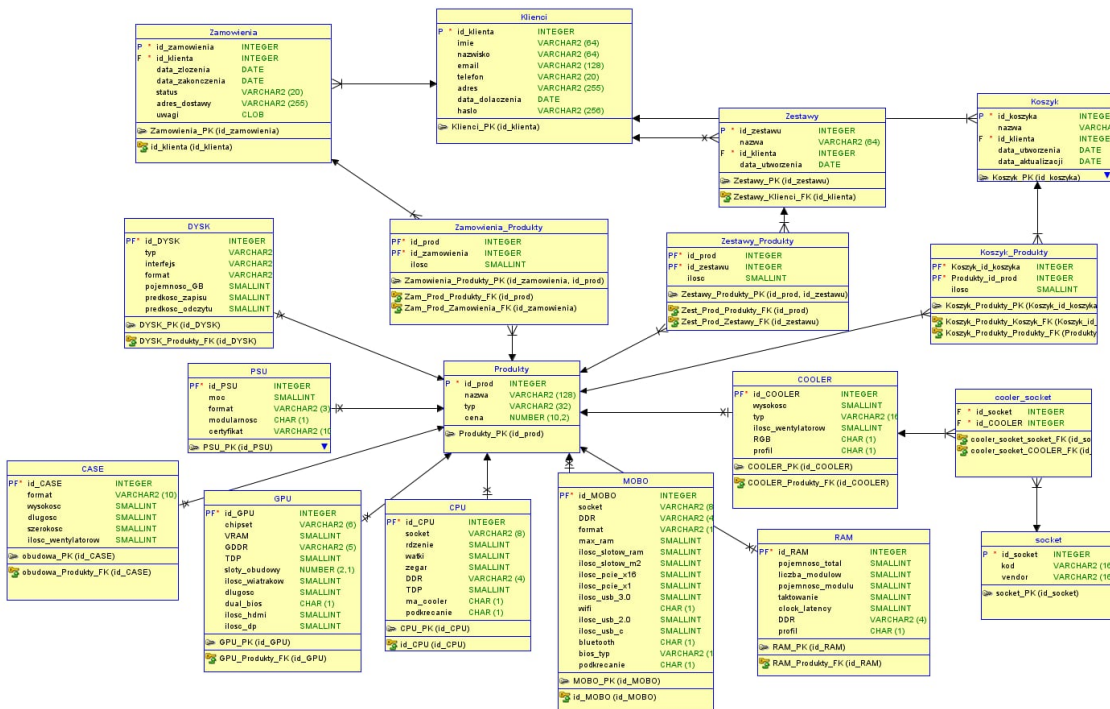
4.2.1 Warstwa frontendowa

4.2.2 Warstwa backendowa

4.2.3 Warstwa danych i infrastruktury

4.3 Opis struktur danych

4.3.1 Model danych - ERD



Rysunek 4.1: Diagram ERD bazy danych

?? przedstawia ...

4.3.2 Opis najważniejszych encji

4.4 Komponenty i moduły systemu

4.4.1 Moduł kompatybilności podzespołów

4.4.2 Moduł logiki rozmytej

4.4.3 Moduł zarządzania koszykiem i zamówieniami

4.5 Najważniejsze algorytmy

4.6 Diagramy UML

4.7 Szczegóły implementacji wybranych elementów

Rozdział 5

Weryfikacja i walidacja

5.1 Cel weryfikacji systemu

5.2 Metody testowania

5.3 Organizacja eksperymentów

5.4 Przypadki testowe

5.5 Wyniki testów

Tabela 5.1: Przykładowe wyniki testów kompatybilności zestawów komputerowych.

ID	Test	Dane wejściowe	Problem	Wynik
1	CPU + płyta główna	Ryzen 5 7600 + B650	Brak	Kompatybilne
2	CPU + płyta główna	i5-12600K + B550	Różne sockety	Niekompatybilne
3	RAM + płyta główna	DDR5 6000 + B460	RAM DDR5 + MOBO DDR4	Niekompatybilne
4	GPU + obudowa	RTX 4070 (330 mm) + obudowa 300 mm	Zbyt długie GPU	Niekompatybilne
5	Zasilacz + CPU/GPU	PSU 400W + RTX 3080	Zbyt mała moc	Niekompatybilne
6	Chłodzenie + socket	Cooler AM4 + CPU LGA1700	Niezgodne typy chłodzenia	Niekompatybilne
7	Zestaw kompletny	CPU + GPU + RAM + Mobo + PSU zgodne	Brak	Kompatybilne
8	Zestaw błędny	3 niezgodności (DDR, TDP, GPU)	Złożona	Niekompatybilne

5.6 Wykryte i usunięte błędy

5.7 Ocena poprawności działania systemu

Rozdział 6

Podsumowanie i wnioski

Celem niniejszego rozdziału jest podsumowanie rezultatów uzyskanych podczas realizacji projektu oraz ocena stopnia spełnienia celów pracy. Dodatkowo przedstawiono wnioski dotyczące dalszego możliwego rozwoju systemu oraz trudności napotkane w trakcie implementacji.

6.1 Realizacja celów pracy

6.2 Ocena otrzymanych rezultatów

6.3 Trudności napotkane podczas realizacji

6.4 Możliwe kierunki dalszego rozwoju

6.5 Wnioski końcowe

Bibliografia

- [1] *Morele.net - sklep komputerowy*. 2025. URL: <https://www.morele.net/> (dostęp 17.11.2025).
- [2] *X-kom - sklep komputerowy*. 2025. URL: <https://www.x-kom.pl/> (dostęp 17.11.2025).

Dodatki

Spis skrótów i symboli

- API interfejs programistyczny aplikacji (ang. *Application Programming Interface*)
- ASGI asynchroniczny interfejs serwera aplikacji (ang. *Asynchronous Server Gateway Interface*)
- ATX standard rozmiaru płyt głównych i obudów komputerowych (ang. *Advanced Technology eXtended*)
- CPU procesor centralny (ang. *Central Processing Unit*)
- CRUD podstawowe operacje na danych: tworzenie, odczyt, modyfikacja, usuwanie (ang. *Create, Read, Update, Delete*)
- CSS kaskadowe arkusze stylów (ang. *Cascading Style Sheets*)
- DDR standard pamięci RAM (*Double Data Rate*)
- ERD diagram związków encji (ang. *Entity–Relationship Diagram*)
- GPU procesor graficzny (ang. *Graphics Processing Unit*)
- HTML język znaczników stron WWW (ang. *HyperText Markup Language*)
- HTTP protokół komunikacji klient-serwer (ang. *HyperText Transfer Protocol*)
- JSON format danych tekstowych (ang. *JavaScript Object Notation*)
- ORM mapowanie obiektowo-relacyjne (ang. *Object–Relational Mapping*)
- PSU zasilacz komputerowy (ang. *Power Supply Unit*)
- RAM pamięć operacyjna (ang. *Random Access Memory*)
- REST styl architektury usług sieciowych (ang. *Representational State Transfer*)
- SQL język zapytań do baz danych (ang. *Structured Query Language*)
- SSD dysk półprzewodnikowy (ang. *Solid State Drive*)

TDP maksymalna ilość ciepła odprowadzana przez układ (ang. *Thermal Design Power*)

TS język programowania TypeScript (ang. *TypeScript*)

UI interfejs użytkownika (ang. *User Interface*)

UML zunifikowany język modelowania (ang. *Unified Modeling Language*)

URL adres zasobu internetowego (ang. *Uniform Resource Locator*)

WWW ogólnosiwiatowa sieć internetowa (ang. *World Wide Web*)

Źródła

Jeżeli w pracy konieczne jest umieszczenie długich fragmentów kodu źródłowego, należy je przenieść w to miejsce.

```
1 if (_nClusters < 1)
2     throw std::string ("unknown number of clusters");
3 if (_nIterations < 1 and _epsilon < 0)
4     throw std::string ("You should set a maximal number of
        iteration or minimal difference — epsilon.");
5 if (_nIterations > 0 and _epsilon > 0)
6     throw std::string ("Both number of iterations and minimal
        epsilon set — you should set either number of iterations
        or minimal epsilon.");
```

Lista dodatkowych plików, uzupełniających tekst pracy

W systemie do pracy dołączono dodatkowe pliki zawierające:

- źródła programu,
- dane testowe,
- film pokazujący działanie opracowanego oprogramowania lub zaprojektowanego i wykonanego urządzenia,
- itp.

Spis rysunków

4.1	Diagram ERD bazy danych	16
-----	-----------------------------------	----

Spis tabel

5.1	Przykładowe wyniki testów kompatybilności zestawów komputerowych. . .	18
-----	---	----